CSC311 - Final Project

Chloe Lam, Sinan Li, Nicholas Poon

Due date: Monday, Dec 4, 2023, 11:59:00 PM EDT

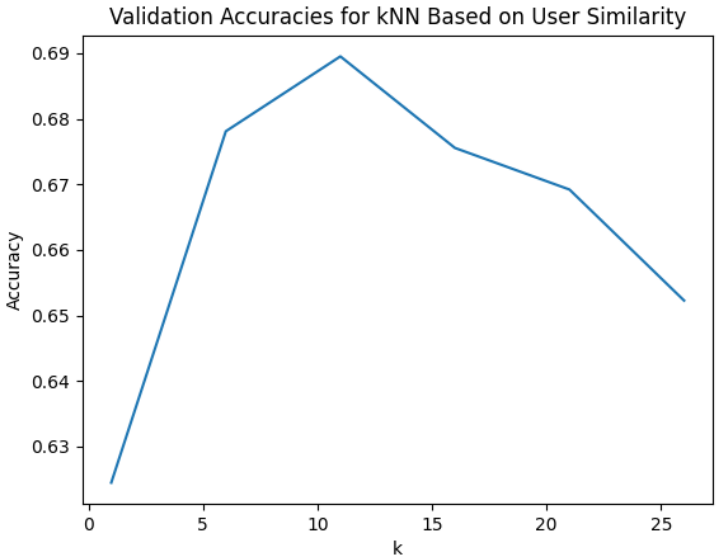
1 Part A

In the first part of the project, you will implement and apply various machine learning algorithms you studied in the course to predict students' correctness of a given diagnostic question. Review the course notes if you don't recall the details of each algorithm. For this part, you will only be using the primary data: `train_data.csv`, `sparse_matrix.npz`, `valid_data.csv`, and `test_data.csv`. Moreover, you may use the helper functions provided in `utils.py` to load the dataset and evaluate your model. You may also use any functions from packages NumPy, Scipy, Pandas, and PyTorch. Make sure you understand the code instead of using it as a black box.

1. **[5pts] k-Nearest Neighbor.** In this problem, using the provided code at `part_a/knn.py`, you will experiment with k-Nearest Neighbor (kNN) algorithm.

   The provided kNN code performs collaborative filtering that uses other students' answers to predict whether the specific student can correctly answer some diagnostic questions. In particular, the starter code implements user-based collaborative filtering: given a user, kNN finds the closest user that similarly answered other questions and predicts the correctness based on the closest student's correctness. The core underlying assumption is that if student A has the same correct and incorrect answers on other diagnostic questions as student B, A's correctness on specific diagnostic questions matches that of student B.

   (a) Complete a function main located at `knn.py` that runs kNN for different values of $k \in \{1, 6, 11, 16, 21, 26\}$. Plot and report the accuracy on the validation data as a function of k.



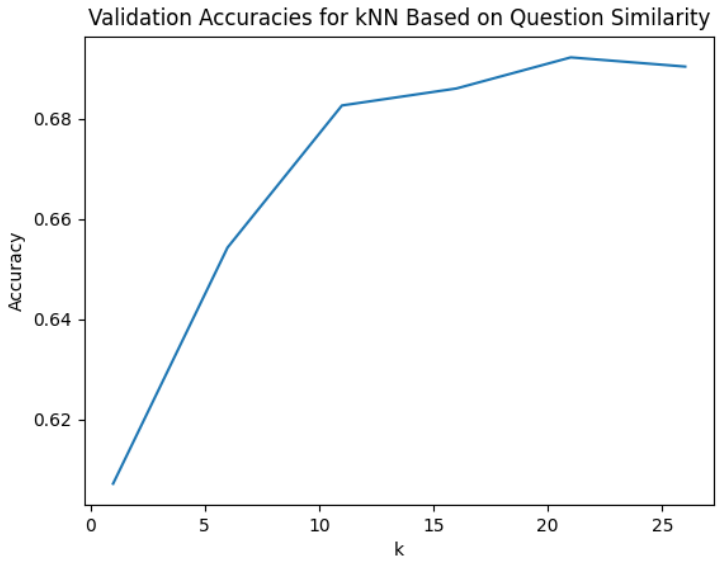
```
Validation Accuracy (k = 1):  0.6244707874682472
Validation Accuracy (k = 6):  0.6780976573525261
Validation Accuracy (k = 11):  0.6895286480383855
Validation Accuracy (k = 16):  0.6755574372001129
Validation Accuracy (k = 21):  0.6692068868190799
Validation Accuracy (k = 26):  0.6522720858029918
```

(b) Choose k^* that has the highest performance on validation data. Report the chosen k^* and the final test accuracy.

```
Test accuracy for kNN based on student similarity with k* = 11:  0.6841659610499576
```

(c) Implement a function `knn_impute_by_item` on the same file that performs item-based collaborative filtering instead of user-based collaborative filtering. Given a question, kNN finds the closest question that was answered similarly, and predicts the correctness based on the closest question's correctness. State the underlying assumption on item-based collaborative filtering. Repeat part (a) and (b) with item-based collaborative filtering.

(a)




```
Validation accuracy (k = 1) based on question similarity:  0.607112616426757
Validation accuracy (k = 6) based on question similarity:  0.6542478125882021
Validation accuracy (k = 11) based on question similarity:  0.6826136042901496
Validation accuracy (k = 16) based on question similarity:  0.6860005644933672
Validation accuracy (k = 21) based on question similarity:  0.6922099915325995
Validation accuracy (k = 26) based on question similarity:  0.69037538808919
```

The underlying assumption on item-based collaborative filtering is that the questions are related.

(b) `Test accuracy for kNN based on question similarity with k* = 21:  0.6816257408975445`

(d) Compare the test performance between user- and item- based collaborative filtering. State which method performs better.

The test accuracy of both filtering methods is quite similar. However, based on the hyperparameter, k, that yields the highest validation accuracy for each respective filtering method, user-based collaborative filtering performs slightly better.

(e) List at least two potential limitations of kNN for the task you are given.

- The assumption that questions are related so that for any given question, its correctness can be predicted based on the closest question's correctness.
- Computational cost. If there are a lot of students and or a lot of questions, it would take a really long time for kNN to iterate through all the data.

2. **[15pts] Item Response Theory.** In this problem, you will implement an Item-Response Theory (IRT) model to predict students' correctness to diagnostic questions.

   The IRT assigns each student an ability value and each question a difficulty value to formulate a probability distribution. In the one-parameter IRT model, β_j represents the difficulty of the question j, and θ_i that represents the i-th students ability. Then, the probability that the question j is correctly answered by student i is formulated as:

$$p(c_{ij} = 1|\theta_i, \beta_j) = \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}$$

   We provide the starter code in `part_a/item_response.py`.

   (a) Derive the log-likelihood $\log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta})$ for all students and questions. Here $\mathbf{C}$ is the sparse matrix. Also, show the derivative of the log-likelihood with respect to θ_i and β_j (Hint: recall the derivative of the logistic model with respect to the parameters).

   - log-likelihood

$$\begin{aligned}
\log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) &= \log L(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) \\
&= \log \left(\prod_{i,j} p(c_{ij}|\theta_j, \beta_j)^{c_{ij}} \cdot (1 - p(c_{ij}|\theta_j, \beta_j))^{1-c_{ij}} \right) \\
&= \sum_{i,j} \left(c_{ij} \log p(c_{ij} = 1|\theta_i, \beta_j) + (1 - c_{ij}) \log(1 - p(c_{ij} = 1|\theta_i, \beta_j)) \right) \\
&= \sum_{i,j} \left(c_{ij} \log \frac{\exp(\theta_i - \beta_i)}{1 + \exp(\theta_i - \beta_i)} + (1 - c_{ij}) \log \left(1 - \frac{\exp(\theta_i - \beta_i)}{1 + \exp(\theta_i - \beta_i)} \right) \right)
\end{aligned}$$

$$\begin{aligned}
\text{Note that } c_{ij} \log \frac{\exp(\theta_i - \beta_i)}{1 + \exp(\theta_i - \beta_i)} &= c_{ij} \log \exp(\theta_i - \beta_j) - c_{ij} \log \left(1 + \exp(\theta_i - \beta_j)\right) \\
&= c_{ij}(\theta_i - \beta_j) - c_{ij} \log \left(1 + \exp(\theta_i - \beta_j)\right)
\end{aligned}$$

$$\begin{aligned}
\text{Note also } (1 - c_{ij}) \log \left(1 - \frac{\exp(\theta_i - \beta_i)}{1 + \exp(\theta_i - \beta_i)} \right) &= (1 - c_{ij}) \log \left(\frac{1}{1 + \exp(\theta_i - \beta_i)} \right) \\
&= (1 - c_{ij})(0 - \log(1 + \exp(\theta_i - \beta_j))) \\
&= (1 - c_{ij}) \log(1 + \exp(\theta_i - \beta_j))
\end{aligned}$$

$$\begin{aligned}
\text{Thus, } \log p(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) &= \sum_{i,j} \left(c_{ij}(\theta_i - \beta_j) - c_{ij} \log(1 + \exp(\theta_i - \beta_j)) - (1 - c_{ij}) \log(1 + \exp(\theta_i - \beta_j)) \right) \\
&= \sum_{i,j} \left(c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j)) \right)
\end{aligned}$$

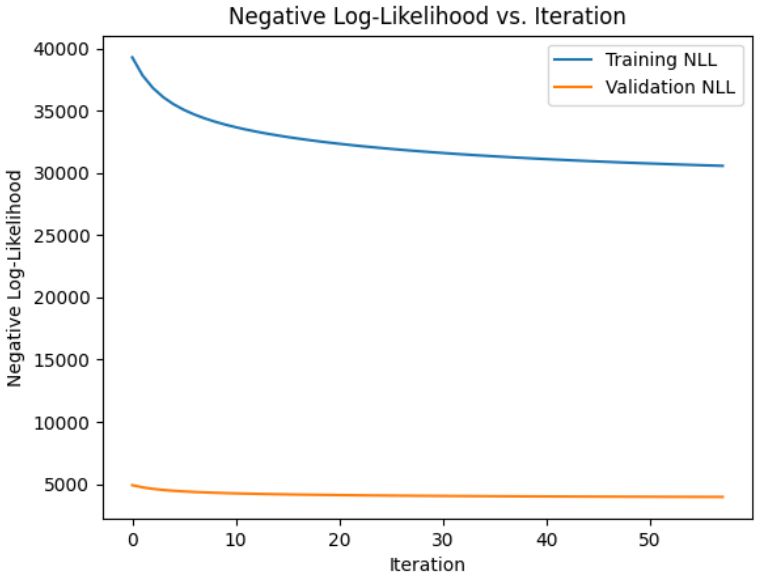
   - Derivative of the log-likelihood

$$\begin{aligned}
\frac{\partial}{\partial \theta_i} \log L(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) &= \frac{\partial}{\partial \theta_i} \sum_{i,j} c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j)) \\
&= \sum_j c_{ij} - \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)} \\
&= \sum_j c_{ij} - p(c_{ij} = 1|\theta_i, \beta_j)
\end{aligned}$$

$$\frac{\partial}{\partial \beta_j} \log L(\mathbf{C}|\boldsymbol{\theta}, \boldsymbol{\beta}) = \frac{\partial}{\partial \beta_j} \sum_{i,j} c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j))$$
$$= \sum_i -c_{ij} - \left(-\frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)} \right)$$
$$= \sum_i p(c_{ij} = 1|\theta_i, \beta_j) - c_{ij}$$

(b) Implement missing functions in `item_response.py` that performs alternating gradient descent on θ and β to maximize the log-likelihood. Report the hyperparameters you selected. With your chosen hyperparameters, report the training curve that shows the training and validation log-likelihoods as a function of iteration.

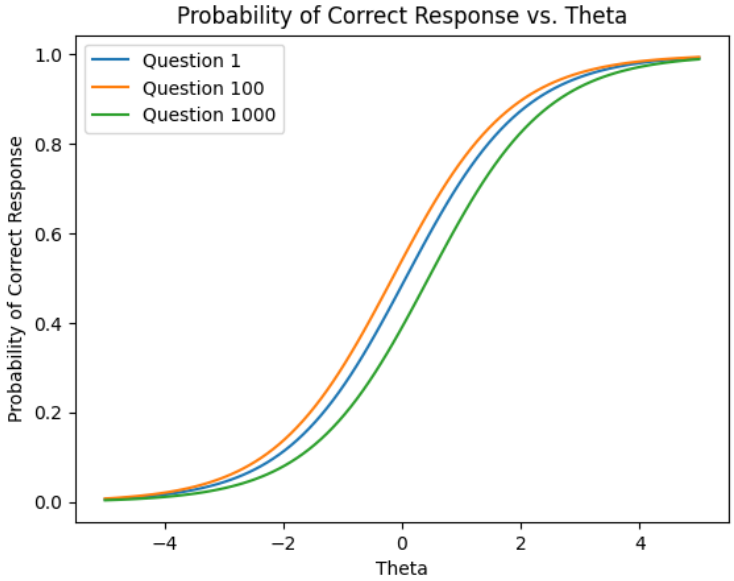
- Learning rate: 0.0025
- Iterations: 58



(c) With the implemented code, report the final validation and test accuracies.

```
Validation Accuracy: 0.7081569291560824
Test Accuracy: 0.703076488851256
```

(d) Select three questions j_1, j_2, and j_3. Using the trained $\boldsymbol{\theta}$ and $\boldsymbol{\beta}$, plot three curves on the same plot that shows the probability of the correct response $p(c_{ij} = 1)$ as a function of θ given a question j. Comment on the shape of the curves and briefly describe what these curves represent.



The curves have the "S" shape, similar to the sigmoid function. This means that

- when θ is low, the probability of having a correct response is also low
- as θ grows, the probability of having a correct response increases rapidly until a certain point
- the increase in probability slows down as θ continues to rise

A curve to the left indicates that the question requires a lower student ability to achieve a higher probability of correctness,

3. **[15pts] Matrix Factorization OR Neural Networks.** In this question, please read both option (i) and option (ii), but you only need to do one of the two.

   (i) **Option 1: Matrix Factorization.** In this problem, you will be implementing matrix factorization methods. The starter code is located at `part_a/matrix_factorization`.

   (a) Using a function `svd_reconstruct` that factorizes the sparse matrix using singular-value decomposition, try out at least 5 different k and select the best k using the validation set. Report the final validation and test performance with your chosen k.

   (b) State one limitation of SVD in the task you are given. (Hint: how are you treating the missing entries?)

   (c) Implement functions `als` and `update_u_z` located at the same file that performs alternating updates. You need to use SGD to update $\mathbf{u}_n \in \mathbb{R}^k$ and $\mathbf{z}_m \in \mathbb{R}^k$ as described in the docstring. To be noted, this is different from the alternating least squares (ALS) we introduced in the course slides, where we used the direct solution. As a reminder, the objective is as follows:

$$\min_{\mathbf{U},\mathbf{Z}} \frac{1}{2} \sum_{(n,m)\in O} \left(C_{nm} - \mathbf{u}_n^\top \mathbf{z}_m\right)^2,$$

   where $\mathbf{C}$ is the sparse matrix and $O = \{(n,m) : \text{entry } (n,m) \text{ of matrix } \mathbf{C} \text{ is observed}\}$.

   (d) Learn the representations $\mathbf{U}$ and $\mathbf{Z}$ using ALS with SGD. Tune learning rate and number of iterations. Report your chosen hyperparameters. Try at least 5 different values of k and select the best k^* that achieves the lowest validation accuracy.

   (e) With your chosen k^*, plot and report how the training and validation squared-error-losses change as a function of iteration. Also report final validation accuracy and test accuracy.

   (ii) **Option 2: Neural Networks.** In this problem, you will implement neural networks to predict students' correctness on a diagnostic question. Specifically, you will design an autoencoder model. Given a user $v \in \mathbb{R}^{N_{\text{questions}}}$ from a set of users $\mathcal{S}$, our objective is:

$$\min_{\boldsymbol{\theta}} \sum_{\mathbf{v}\in\mathcal{S}} \|\mathbf{v} - f(\mathbf{v};\boldsymbol{\theta})\|_2^2,$$

   where f is the reconstruction of the input $\mathbf{v}$. The network computes the following function:

$$f(\mathbf{v};\boldsymbol{\theta}) = h(\mathbf{W}^{(2)} g(W^{(1)}\mathbf{v} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}) \in \mathbb{R}^{N_{\text{questions}}}$$

   for some activation functions h and g. In this question, you will be using sigmoid activation functions for both. Here, $\mathbf{W}^{(1)} \in \mathbb{R}^{k\times N_{\text{questions}}}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{N_{\text{questions}}\times k}$, where $k \in \mathbb{N}$ is the latent dimension. We provide the starter code written in PyTorch at `part_a/neural_network`.

   (a) Describe at least three differences between ALS and neural networks.

   - Since ALS uses a matrix, it assumes a linear relationship in the data. However, neural networks can model complex data, including both linear and nonlinear data.
   - In ALS, there are 2 matrices and optimization is done by fixing 1 matrix and optimizing the other, for both matrices. In neural networks, optimization is done with backpropagation, which involves using the chain rule for gradient descent for each layer.
   - Neural networks typically have activation functions while ALS does not have this.
   - ALS is generally more scalable for large dataset since it can be efficiently parallelized during computing. Neural networks, on the other hand, can require significantly more computational resources, especially when there are many layers and parameters.

   (b) Implement a class `AutoEncoder` that performs a forward pass of the autoencoder following the instructions in the docstring.

   (c) Train the autoencoder using latent dimensions of $k \in \{10, 50, 100, 200, 500\}$. Also, tune optimization hyperparameters such as learning rate and number of iterations. Select k^* that has the highest validation accuracy.

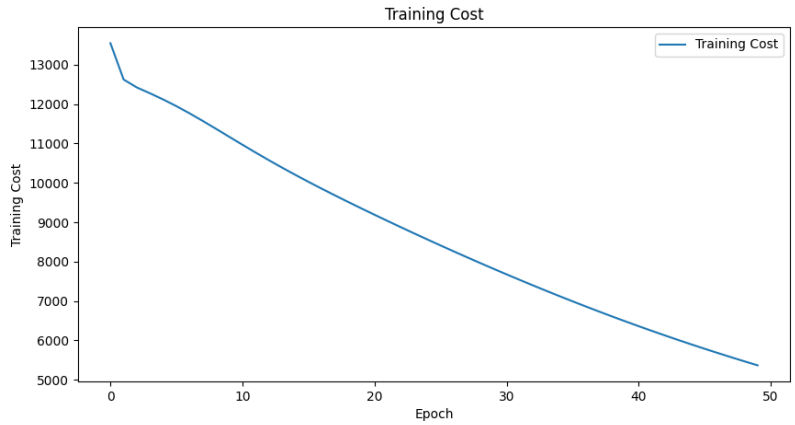
With a learning rate of 0.01, an epoch of 50, and no regularization, we observe the following:

```
Validation accuracy with k = 10:  0.6836014676827548
Validation accuracy with k = 50:  0.6858594411515665
Validation accuracy with k = 100:  0.6878351679367768
Validation accuracy with k = 200:  0.6793677674287327
Validation accuracy with k = 500:  0.6747106971493085
```

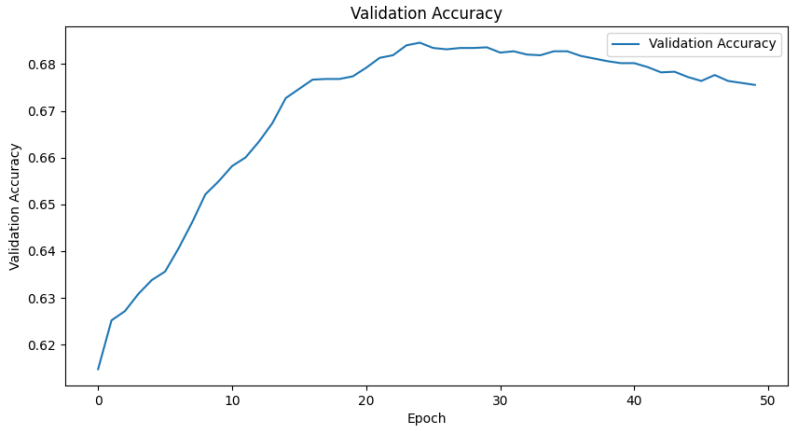
Thus, we choose $k^* = 100$ as it has the highest validation accuracy.

(d) With your chosen k^*, plot and report how the training and validation objectives changes as a function of epoch. Also, report the final test accuracy.

(a)



(b)



The final test accuracy is 0.6664. We observe from the plots that the training loss gets minimized and the validation accuracy gets maximized as we increase epochs.

(e) Modify a function train so that the objective adds the L2 regularization. The objective is as follows:

$$\min_{\boldsymbol{\theta}} \sum_{\mathbf{v} \in \mathcal{S}} \|\mathbf{v} - f(\mathbf{v}; \boldsymbol{\theta})\|_2^2 + \frac{\lambda}{2}(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}\|_F^2)$$

You may use a method `get_weight_norm` to obtain the regularization term. Using the k and other hyperparameters selected from part (d), tune the regularization penalty $\lambda \in \{0.001, 0.01, 0.1, 1\}$. With your chosen λ, report the final validation and test accuracy. Does your model perform better with the regularization penalty?

```
Validation accuracy with λ = 0.001:  0.6784
Validation accuracy with λ = 0.01:  0.6804
Validation accuracy with λ = 0.1:  0.6255
Validation accuracy with λ = 1:  0.6248
```

Thus, we choose $\lambda^* = 0.01$ as it has the highest validation accuracy.
The final validation accuracy is 0.6791. The final test accuracy is 0.6777.
We observe that both validation and test accuracies are slightly higher with the regularization penalty, so we conclude that the model does perform better with the regularization penalty.

4. **[15pts] Ensemble.** In this problem, you will be implementing bagging ensemble to improve the stability and accuracy of your base models. Select and train 3 base models with bootstrapping the training set. You may use the same or different base models. Your implementation should be completed in `part_a/ensemble.py`. To predict the correctness, generate 3 predictions by using the base model and average the predicted correctness. Report the final validation and test accuracy. Explain the ensemble process you implemented. Do you obtain better performance using the ensemble? Why or why not?

   Chosen base model: Item Response Theory (`IRT`)

   Ensemble process:

   - **Bootstrap Sampling:** We created 3 bootstrap samples from the training data. In each bootstrap sample, we randomly selected data points with replacement, and the bootstrap sample has the same size as the original data set.
   - **Training:** For each bootstrap sample, train the base model on the sample.
   - **Prediction Aggregation:** With the three models trained in the previous step, make predictions on the validation data. This will yield three lists of predictions with **True** or **False** as individual entries. For each entry we select the majority class of the three predictions. Notice that no tie breaking condition is required as we have an odd number of predictions, meaning that a majority class is guaranteed to be generated.
   - **Evaluation:** Using the final aggregated prediction from the previous step, calculate the accuracy.

   Final validation and test accuracy:

   ```
   Average Validation Accuracy: 0.6999247342177063
   Average Test Accuracy: 0.6970552262677581

   Ensemble Validation Accuracy: 0.7039232289020604
   Ensemble Test Accuracy: 0.7011007620660458
   ```

   Analysis:

   Compared to the the individual base models, the ensembled model performs slightly better, with a difference in accuracy of less than 0.5%. This phenomenon is due to the fact that bagging ensemble reduces variance in the model and prevents overfitting.

   Compared to the outcome from Question 2, the performance did not improve. As we can see in the difference between our original validation and test accuracy in Question 2, our model was not overfitting to begin with. The variance was large enough to perform well on the test data but small enough to prevent generalizing to the validation data. Thus, it was not necessary to reduce variance to prevent overfitting. In this case, generalizing the data does not improve the accuracy of the model. Furthermore, the capacity of the base model may be capped at approximately 70%; thus a more powerful base model could improve the performance.

2 Part B

In the second part of the project, you will modify one of the algorithms you implemented in part A to hopefully predict students' answers to the diagnostic question with higher accuracy. In particular, consider the results obtained in part A, reason about what factors are limiting the performance of one of the methods (e.g. overfitting? underfitting? optimization difficulties?) and come up with a proposed modification to the algorithm which could help address this problem. Rigorously test the performance of your modified algorithm, and write up a report summarizing your results as described below.

You will not be graded on how well the algorithm performs (i.e. its accuracy); rather, your grade will be based on the quality of your analysis. Try to be creative! You may also optionally use the provided metadata (`question_meta.csv` and `student_meta.csv`) to improve the accuracy of the model. At last, you are free to use any third-party ideas or code as long as it is publicly available. You must properly provide references to any work that is not your own in the write-up.

The length of your report for part B should be 3-4 pages. Don't be afraid to keep the text short and to include large illustrative figures. The guidelines and marking schemes are as follows:

1. **[15pts] Formal Description:** Precisely define the way in which you are extending the algorithm. You should provide equations and possibly an algorithm box. Describe why your proposed method should be expected to perform better. For instance, are you intending it to improve the optimization, reduce overfitting, etc.?

2. **[10pts] Figure or Diagram:** that shows the overall model or idea. The idea is to make your report more accessible, especially to readers who are starting by skimming your report.

3. **[15pts] Comparison or Demonstration:** Include:

    - A comparison of the accuracies obtained by your model to those from baseline models. Include a table or a plot for an illustrative comparison.
    - Based on the argument you gave for why your extension should help, design and carry out an experiment to test your hypothesis. E.g., consider how you would disentangle whether the benefits are due to optimization or regularization.

4. **[15pts] Limitations:** of your approach.

    - Describe some settings in which we'd expect your approach to perform poorly, or where all existing models fail.
    - Try to guess or explain why these limitations are the way they are.
    - Give some examples of possible extensions, ways to address these limitations, or open problems.

2.1 Formal Description

2.1.1 Limitations of IRT

In the original Item Response Theory (IRT) model, only the ability of students ($\boldsymbol{\theta}$) and the difficulty of questions ($\boldsymbol{\beta}$) were taken into consideration. While this model effectively captures the basic dynamics of the students' performance, it has several limitations.

- **Lack of Contextual Factors**

  The existing model does not account for external factors, for example, the subject-specific challenges for student demographics, which may impact the student performance.

- **Treating Questions Homogeneously**

  The existing model treat all questions equally without considering their subject matter, while in reality different students may have different performances on different subjects on mathematics.

- **Overlooking Student Diversity**

  The IRT model does not consider the diferse background of students, which may have significant influences on their capabilities. For example, having the same capability, a student of age 17 would be more likely to answer a more difficult question, compared to a student of age 8.

- **Linear Model**

  The original IRT is a linear IRT model, which may experience difficulties expressing more complex patterns.

2.1.2 Extending IRT

To overcome these limitations, we introduce two additional features: the subject specific effects ($\boldsymbol{\gamma}$), and student demographic features ($\boldsymbol{\delta}$). This Extended Item Response Theory (EIRT) model aimes to enhance prediction accuracy by reducing bias and underfitting.

2.1.3 The EIRT Model

The probability of a correct answer in the EIRT model is given by

$$p(c_{ij} = 1|\theta_i, \beta_j, \boldsymbol{\gamma}_i, \boldsymbol{\delta}_i) = \frac{\exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_i \cdot \mathbf{s}_j)}{1 + \exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_i \cdot \mathbf{s}_j)}$$

where

- θ_i is the ability of student i
- β_j is the difficulty of question j
- $\boldsymbol{\gamma}_i$ is the adjustment factor of the subjects for student i
- $\boldsymbol{\delta}_i$ is the demographic of student i
- $\mathbf{s}_j$ is a binary vector for the subject of questions j
- $\mathbf{m}_j$ is the numerical representation of the student's demographic

We use the same loss function (negative log-likelihood) and optimization algorithm (gradient descent) as the original IRT model training process.

The loss function, $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\beta}, \boldsymbol{\gamma}, \boldsymbol{\delta})$, can be derived from

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\beta}, \boldsymbol{\gamma}, \boldsymbol{\delta}) &= -\sum_{i,j} \left(c_{ij} \log \left(p(c_{ij} = 1 | \theta_i, \beta_j, \boldsymbol{\gamma}_i, \boldsymbol{\delta}_i)\right) + (1 - c_{ij}) \log \left(1 - p(c_{ij} = 1 | \theta_i, \beta_j, \gamma_j, \boldsymbol{\delta}_i)\right)\right) \\
&= -\sum_{i,j} \left(c_{ij} \log \left(\frac{\exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_{ik} \cdot \mathbf{m}_j)}{1 + \exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_{ik} \cdot \mathbf{m}_j)}\right) + \right. \\
&\qquad \left. (1 - c_{ij}) \log \left(1 - \frac{\exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_{ik} \cdot \mathbf{m}_j)}{1 + \exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_{ik} \cdot \mathbf{m}_j)}\right)\right) \\
&= -\sum_{i,j} \left(c_{ij} \left(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_{ik} \cdot \mathbf{m}_j\right) - \log \left(1 + \exp(\theta_i - \beta_j + \boldsymbol{\gamma}_i \cdot \mathbf{s}_j + \boldsymbol{\delta}_{ik} \cdot \mathbf{m}_j)\right)\right)
\end{aligned}$$

For gradient descent, we compute the derivative of the log-likelihood with respective to the parameters.

- $\dfrac{\partial \mathcal{L}}{\partial \theta_i} = \sum_j \left(c_{ij} - p(c_{ij} = 1 | \theta_i, \beta_j, \gamma_j, \boldsymbol{\delta}_i)\right)$
- $\dfrac{\partial \mathcal{L}}{\partial \beta_j} = \sum_j \left(p(c_{ij} = 1 | \theta_i, \beta_j, \gamma_j, \boldsymbol{\delta}_i) - c_{ij}\right)$
- $\dfrac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_j \left(c_{ij} - p(c_{ij} = 1 | \theta_i, \beta_j, \gamma_j, \boldsymbol{\delta}_i)\right) \mathbf{s}_j$
- $\dfrac{\partial \mathcal{L}}{\partial \delta_{ik}} = \sum_j \left(c_{ij} - p(c_{ij} = 1 | \theta_i, \beta_j, \gamma_j, \boldsymbol{\delta}_i)\right) \mathbf{m}_j$

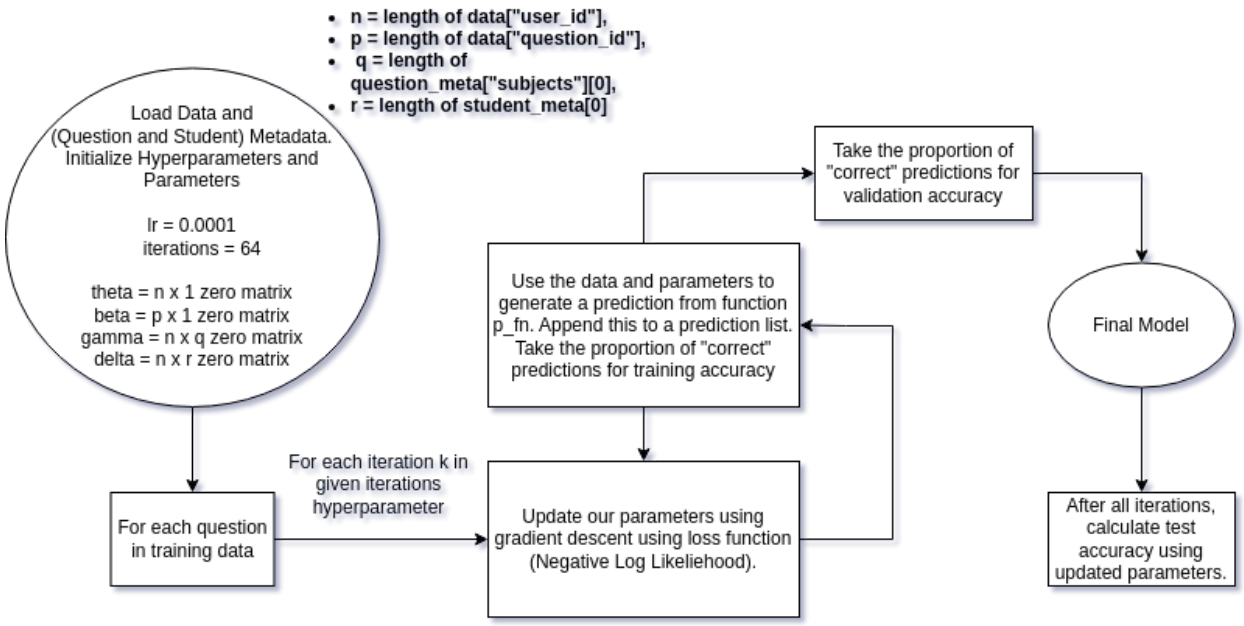


Figure 1: Overall model process

2.2 Comparison or Demonstration

Given that the extended model with additional parameters like subject adjustment factors ($\boldsymbol{\gamma}_i$) and demographics ($\boldsymbol{\delta}_i$), we hypothesize that these additions captures more variability in student performance, thus improving accuracy. To test this hypothesis, we can design an experiment with the following steps:

- **Model Comparison**: Train the original IRT model (using only θ_i and β_j) and the extended model on the same training set and evaluate them on the same validation set.
- **Overfitting Check**: Compare training and validation losses for both models. If the extended model's validation loss is significantly higher than its training loss, this suggests overfitting.
- **Optimization Check**: Monitor the convergence of both models during training. If the extended model's parameters do not stabilize, or if the training loss does not decrease smoothly, there may be optimization issues.
- **Feature Analysis**: Evaluate the importance of the added features ($\boldsymbol{\gamma}_i$ and $\boldsymbol{\delta}_i$). If they do not contribute significantly to the model's predictions, consider simplifying the model.

	IRT	EIRT	EIRT (SGD)	EIRT (SGD Dynamic LR)
Validation	70.81%	68.68%	68.56%	68.68%
Test	70.31%	66.10 %	67.54%	67.63%

Table 1: Validation and Test Accuracy for IRT and EIRT

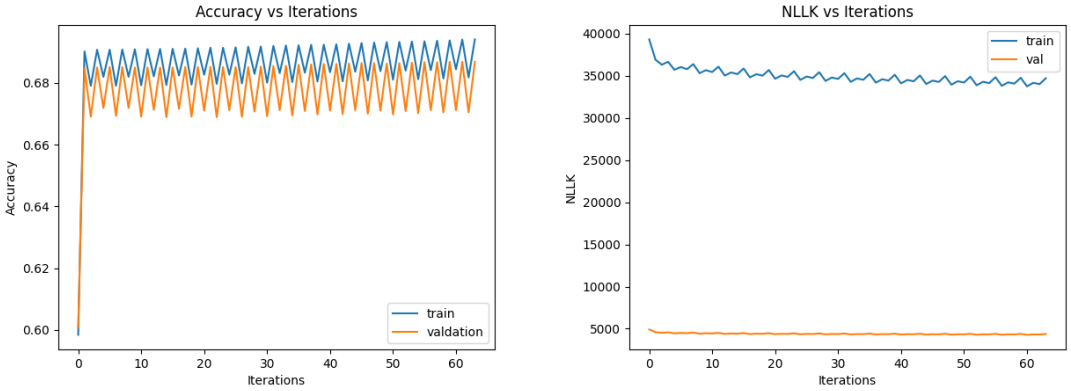


Figure 2: Accuracy and Loss for Training and Validation, EIRT

The observed performance drop in Table 1 from the original IRT model to the EIRT model can potentially be attributed to several factors. The oscillatory pattern observed in Figure 2 for both training and validation suggests instability in the learning process. Initially, such fluctuations were thought to be caused by high learning rates, which typically leads to the model parameters overshooting optimal values during updates. However, such oscillatory behaviour persisted even after reducing the learning rate to a conservative value of 0.0001, indicating the problem lying somewhere else.

We further hypothesized that the additional parameters that account for subject adjustments ($\boldsymbol{\gamma}_i$) and demographics ($\boldsymbol{\delta}_i$) increase the dimensionality of the parameter space, which can make the optimization landscape more prone to local minima or saddle points, complicating the convergence of gradient-based optimization methods. We predict that the extra dimensions is causing our model to become more sensitive as we were previously trying to represent these parameters in a linear space, which is equivalent to projecting our dataset onto a lower dimension.

What we noticed is that with batch gradient descent, there was a lower validation and test accuracy than the base model. This suggests that this extended model is actually underfitting to the data. As a result,

we wanted to increase the variance to try to remove this underfitting tendency. Therefore, we decided to try stochastic gradient descent (SGD), as it tends to have a higher variance compared to Batch Gradient Descent (BGD). After doing this, we realized that the validation and test accuracy improved slightly from the BGD model but it is still less than our original model.

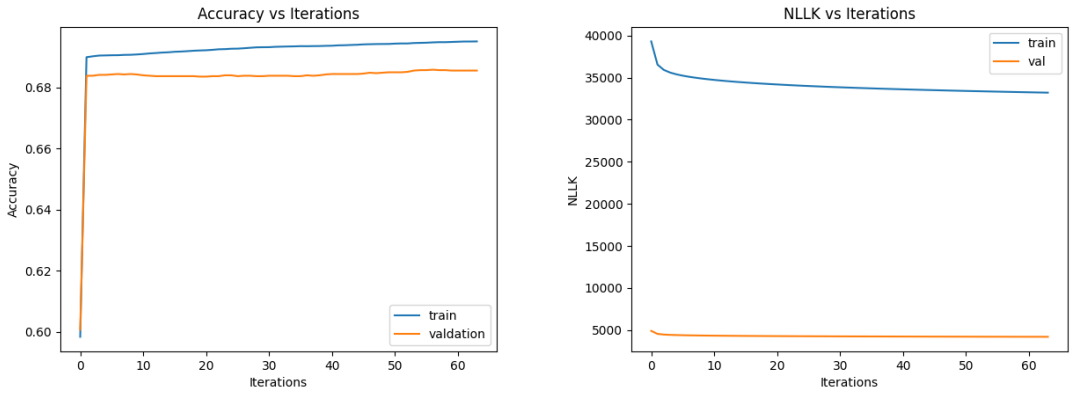


Figure 3: Accuracy and Loss for Training and Validation, EIRT with SGD

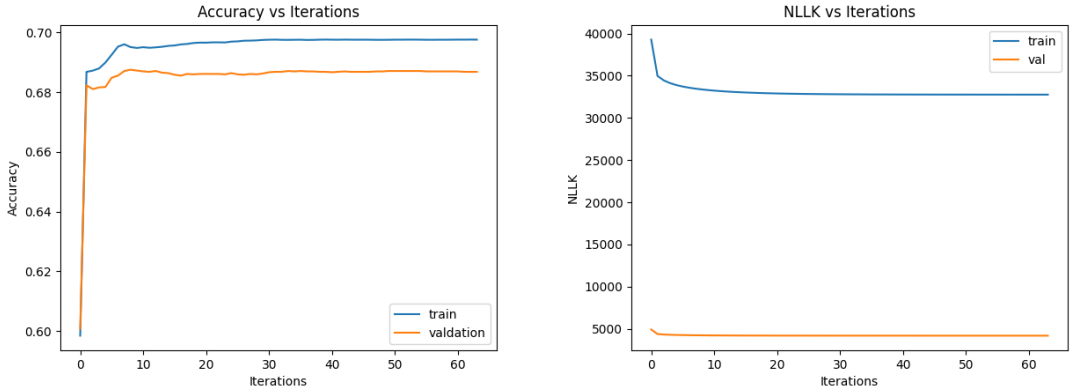


Figure 4: Accuracy and Loss for Training and Validation, EIRT (SGD) with dynamic learning rate

2.3 Limitations

- In our model, we are working under the assumption that all questions are independent of each other. Thus, given a student and an unanswered question, the model looks to see how the student answered all of the other questions to predict the student's ability to answer the current question correctly. However, to achieve a higher accuracy, the model should really be looking at how the student answered all of the other relevant questions. This would be more accurate as given the student's response to another relevant question, we can confirm what the student knows or doesn't know about the subject, which would affect their ability to answer the given question.

- It is also important to note that there are instances where the student has answered a question more than once. Although the data is not able to capture this information as it only saves the most recent response, our model is not able to take recency into account either. When a student answers a question more than once, they have learned from the previous time that they have answered the same question. Specifically, they may have learned more information about the topic, which would enhance their ability

to answer the question. If a student is able to adapt to their changing environment, the model should be able to adapt as well. This suggests that using some sort of reinforcement learning algorithm may actually be beneficial in this situation.

- Our model also has the underlying assumption that students will use only use the best of their knowledge to answer a question. We assume there is a correlation between a student's ability in a certain subject with their ability to answer the question. It does not take into account that a student may know nothing or not enough about the topic and make a guess on what the correct answer is, since the questions are multiple choice questions.

3 Contributions

- Part A Question 1
  - Nicholas
- Part A Question 2
  - Lance
- Part A Question 3
  - Chloe
- Part A Question 4
  - Chloe
  - Lance
  - Nicholas
- Part B
  - Chloe
  - Lance
  - Nicholas

Note: All of us completed the course evaluation upon submission of this assignment.

4 References

[1] Zichao Wang et al. *Results and Insights from Diagnostic Questions: The NeurIPS 2020 Education Challenge*. 2021. arXiv: 2104.04034 [cs.CY].